

Logic

April 13, 2026

0.1 Notes

1. The notebooks are largely self-contained, i.e, if you see a symbol there will be an explanation about it at some point in the notebook.
 - Most often there will be links to the cell where the symbols are explained
 - If the symbols are not explained in this notebook, a reference to the appropriate notebook will be provided
2. **Github does a poor job of rendering this notebook.** The online render of this notebook is missing links, symbols, and notations are badly formatted. It is advised that you clone a local copy (or download the notebook) and open it locally.

1 Contents

1. Logic
 - Boolean operations
 - And
 - Or
 - Not
 - Exclusive Or
 - Nand
 - Proof symbols
 - Implies
 - Implied by
 - If and only if
 - Therefore
 - Because
 - Contradiction
 - End of Proof
 - Quantifiers
 - For all
 - There exists
 - There exists uniquely
 - Combining quantifiers

1.1 Importing Libraries

```
[1]: import random
import math
```

Boolean Operations

And

The Boolean And operation is denoted using $\wedge$

$$p \wedge q$$

```
[2]: boo = [True, False]

print('p AND q truth table\n')
for p in boo:
    for q in boo:
        print(f'p: {str(p):5s}  q: {str(q):5s}  p and q: {p and q}')
```

p AND q truth table

p: True	q: True	p and q: True
p: True	q: False	p and q: False
p: False	q: True	p and q: False
p: False	q: False	p and q: False

Or

The Boolean Or operation is denoted using $\vee$

$$p \vee q$$

```
[3]: boo = [True, False]

print('p OR q truth table\n')
for p in boo:
    for q in boo:
        print(f'p: {str(p):5s}  q: {str(q):5s}  p or q: {p or q}')
```

p OR q truth table

p: True	q: True	p or q: True
p: True	q: False	p or q: True
p: False	q: True	p or q: True
p: False	q: False	p or q: False

Not

The Boolean Not operation is denoted using $\sim$ or $\neg$ and sometimes just the text ‘not’ is used:

$$\sim p$$

or

$$\neg p$$

or

$$\text{not } p$$

```
[4]: boo = [True, False]

print('NOT p truth table\n')
for p in boo:
    print(f'p: {str(p):5s} not p: {not p}')
```

NOT p truth table

```
p: True   not p: False
p: False  not p: True
```

Exclusive Or

The Exclusive Or operation is denoted using $\underline{\vee}$ or $\oplus$ or just XOR

$$p \underline{\vee} q$$

or

$$p \oplus q$$

or

$$p \text{ XOR } q$$

```
[5]: boo = [True, False]

print('p XOR q truth table\n')
for p in boo:
    for q in boo:
        print(f'p: {str(p):5s} q: {str(q):5s} p xor q: {p ^ q}')
```

p XOR q truth table

```
p: True   q: True   p xor q: False
p: True   q: False  p xor q: True
p: False  q: True   p xor q: True
p: False  q: False  p xor q: False
```

Nand

The Nand operation is denoted using $\bar{\wedge}$

$$p \bar{\wedge} q$$

The Nand operator can be expanded to: $p \bar{\wedge} q = \neg(p \wedge q)$

```
[6]: boo = [True, False]

print('p NAND q truth table\n')
for p in boo:
    for q in boo:
        print(f'p: {str(p):5s}  q: {str(q):5s}  p nand q: {not (p and q)}')
```

p NAND q truth table

```
p: True   q: True   p nand q: False
p: True   q: False  p nand q: True
p: False  q: True   p nand q: True
p: False  q: False  p nand q: True
```

Proof Symbols

Implies

In mathematical proofs, the term ‘implies’ means ‘if a then b ’ or ‘ a implies b ’ and this is denoted using the symbol: $\Rightarrow$

$$a \Rightarrow b$$

Here a and b are any mathematical concepts (or *logical predicates*)

Logically, $a \Rightarrow b$ is equivalent to $b \vee \neg a$

(See: Or)

```
[7]: boo = [True, False]
print('a implies b truth table\n')
for a in boo:
    for b in boo:
        print('a: ', a, 'b: ', b, ', a implies b :', b or (not a))
```

a implies b truth table

```
a: True b: True , a implies b : True
a: True b: False , a implies b : False
```

a: False b: True , a implies b : True
a: False b: False , a implies b : True

To prove a theorem of this form, you must prove that b is true whenever a is true. Example: if x is greater than or equal to 4, then $2^x \geq x^2$

$$\forall x \in \mathbb{R}, x \geq 4 \Rightarrow 2^x \geq x^2$$

i.e: if $a = (x \geq 4)$, then $b = (2^x \geq x^2)$

Notice from the above truth table that a may be False when b is True. However, b **must** be True when a is True. Hence, we can prove that b is True whenever a is True for $a \Rightarrow b$ but also, if we show that b is False when a is True, we have invalidated the $a \Rightarrow b$ statement.

(See: For all)

For more information on the real set and the belongs to notation see the Collections notebook

```
[8]: print('if a then b')

X = [-10.00,-2.20,0.00,2.00,3.10,4.00,5.5,6.00,7.8] #Subset of R used as an example

a_implies_b = []

for x in X:
    condition_a = x >= 4
    condition_b = 2**x >= x**2
    print('x :', x, ', x>=4, a: ', condition_a, ', 2^x>=x^2, b: ', condition_b)

    a_implies_b.append(condition_b or (not condition_a))

print('\nCompared with the truth table above')
print('a implies b: ', all(a_implies_b))
#Atleast for this subset of R
```

```
if a then b
x : -10.0 , x>=4, a: False , 2^x>=x^2, b: False
x : -2.2 , x>=4, a: False , 2^x>=x^2, b: False
x : 0.0 , x>=4, a: False , 2^x>=x^2, b: True
x : 2.0 , x>=4, a: False , 2^x>=x^2, b: True
x : 3.1 , x>=4, a: False , 2^x>=x^2, b: False
x : 4.0 , x>=4, a: True , 2^x>=x^2, b: True
x : 5.5 , x>=4, a: True , 2^x>=x^2, b: True
x : 6.0 , x>=4, a: True , 2^x>=x^2, b: True
x : 7.8 , x>=4, a: True , 2^x>=x^2, b: True
```

Compared with the truth table above
a implies b: True

Implied by

In mathematical proofs, the term ‘implied by’ means ‘if b then a ’ or ‘ a is implied by b ’ and this is denoted using the symbol: $\Leftarrow$

$$a \Leftarrow b$$

Here a and b are any mathematical concepts (or *logical predicates*)

Logically, $a \Leftarrow b$ is equivalent to $a \vee \neg b$

(See: Or)

```
[9]: boo = [True, False]
print('a implied by b truth table\n')
for a in boo:
    for b in boo:
        print('a: ', a, 'b: ', b, ', a implied by b :', a or (not b))
```

a implied by b truth table

```
a: True b: True , a implied by b : True
a: True b: False , a implied by b : True
a: False b: True , a implied by b : False
a: False b: False , a implied by b : True
```

To prove a theorem of this form, you must prove that a true whenever b is true. To explain the concept, let's expand on the previous example, but here let's assume that the opposite condition is true, i.e: $x \geq 4$ is implied by $2^x \geq x^2$

$$\forall x \in \mathbb{R}, x \geq 4 \Leftarrow 2^x \geq x^2$$

So based on our truth table above we have: $a = (x \geq 4)$, $b = (2^x \geq x^2)$. Interestingly, if this is true we would have proved that $x \geq 4$ if and only if $2^x \geq x^2$ (See: if and only if)

(See: For all)

For more information on the real set and the belongs to notation see the Collections notebook

Now based on the truth table above if we observe a is False when b is True we have essentially disproved the implied by assertion:

```
[10]: print('if b then a')

X = [0.00, 2.00] #Subset of R used as an example

a_implied_by_b = []

for x in X:
```

```

condition_a = x >= 4
condition_b = 2**x >= x**2
print('x :', x, ', 2^x>=x^2, b: ', condition_b, ', x>=4, a: ', condition_a,)

a_implied_by_b.append(condition_a or (not condition_b))

print('\nCompared with the truth table above')
print('a implied by b: ', all(a_implied_by_b))

```

```

if b then a
x : 0.0 , 2^x>=x^2, b: True , x>=4, a: False
x : 2.0 , 2^x>=x^2, b: True , x>=4, a: False

```

Compared with the truth table above
a implied by b: False

If and only if

In mathematical proofs, the term ‘if and only if’ means ‘if a then b ’ as well as ‘if b then a ’ and this is denoted using the symbol: $\iff$ but it is also sometimes abbreviated as: iff

$$a \iff b$$

or

$$a \text{ iff } b$$

The concept of iff is also logically equivalent to $(a \Rightarrow b) \wedge (b \Rightarrow a)$

(See: And and Implies)

Here a and b are any mathematical concepts (or *logical predicates*).

```

[11]: boo = [True, False]
print('a iff b truth table\n')
for a in boo:
    for b in boo:
        print('a: ', a, 'b: ', b, ', a iff b: ', (b or (not a)) and (a or (not
        b)))

```

a iff b truth table

```

a: True b: True , a iff b : True
a: True b: False , a iff b : False
a: False b: True , a iff b : False
a: False b: False , a iff b : True

```

To prove a theorem of this form, you must prove that a and b are equivalent. Not only is b true whenever a is true, but a is true whenever b is true. Example: The integer n is odd if and only if n^2 is odd.

$$\forall n \in \mathbb{Z}, n \text{ is odd} \iff n^2 \text{ is odd}$$

i.e: $(a = n \text{ is odd}) \text{ iff } (b = n^2 \text{ is odd})$

(See: For all)

For more information on the integer set and the belongs to notation see the Collections notebook

```
[12]: def is_odd(n):
      return n % 2 != 0

      # a: n is odd, b: n^2 is odd
      # Test a iff b: (a implies b) and (b implies a)
      N = [-3, -2, -1, 0, 1, 2, 3, 4, 5]
      a_iff_b = []

      for n in N:
          a = is_odd(n)
          b = is_odd(n ** 2)
          result = (a == b)
          a_iff_b.append(result)
          print(f'n={n:2}, odd(n)={str(a):5s}, odd(n^2)={str(b):5s}, a iff b:␣
↪{result}')

      print(f'\na iff b holds for all n: {all(a_iff_b)}')
```

```
n=-3, odd(n)=True , odd(n^2)=True , a iff b: True
n=-2, odd(n)=False, odd(n^2)=False, a iff b: True
n=-1, odd(n)=True , odd(n^2)=True , a iff b: True
n= 0, odd(n)=False, odd(n^2)=False, a iff b: True
n= 1, odd(n)=True , odd(n^2)=True , a iff b: True
n= 2, odd(n)=False, odd(n^2)=False, a iff b: True
n= 3, odd(n)=True , odd(n^2)=True , a iff b: True
n= 4, odd(n)=False, odd(n^2)=False, a iff b: True
n= 5, odd(n)=True , odd(n^2)=True , a iff b: True
```

a iff b holds for all n: True

Therefore

The term *therefore* is denoted by: $\therefore$

$$r^2 + \lambda^2 c^2 = 0$$

$$\therefore r = \pm \lambda ci$$

Because

The term *because* is denoted by: $\because$

$$\forall x + 1 = 10 \forall x = 9$$

Contradiction

Contradiction in a proof is denoted by: $\Rightarrow \Leftarrow$

Used to show that the supposition was False

End of Proof

The end of a proof is show using the following notations or text:

Just a square box:



a filled square box:



or the text:

QED

Quantifiers

For all

Also called a universal quantifier. The ‘for all’ symbol is used simply to denote that a concept or relation (or *logical predicates*) is applied to every member of the domain. Denoted by $\forall$

For example: squares of all real numbers are positive or zero can be expressed through:

$$\forall x \in \mathbb{R}, x^2 \geq 0$$

Which can be read as, for all x belonging to the set of real numbers (essentially any real number), the square of x is always greater or equal to zero.

For more information on the real set and the belongs to notation see the Collections notebook

```
[13]: # for all x in R, x^2 >= 0
X = [random.uniform(-1000, 1000) for _ in range(5)]

for x in X:
    print(f'x = {x:8.2f}, x^2 = {x**2:12.2f}, x^2 >= 0: {x**2 >= 0}')
```

```
x = 728.51, x^2 = 530731.84, x^2 >= 0: True
x = -843.56, x^2 = 711596.87, x^2 >= 0: True
x = -779.40, x^2 = 607464.14, x^2 >= 0: True
x = -434.49, x^2 = 188779.24, x^2 >= 0: True
x = 641.31, x^2 = 411283.24, x^2 >= 0: True
```

The for all $\forall$ notations can be extended to denote complex statements. For example the commutative property of addition can be denoted using:

$$\forall x \in \mathbb{R}, \forall y \in \mathbb{R}, x + y = y + x$$

There exists

Also called an existential quantifier. This symbol can be interpreted as ‘there exists’, ‘there is at least one’, or ‘for some’ and applied to a mathematical concept (or *logical predicates*); it is denoted by: $\exists$

For example: there exists at least one real number x whose square equals 2

$$\exists x \in \mathbb{R}, x^2 = 2$$

This can be read as there exists at least one x belonging to the real set of numbers such that $x^2 = 2$. With this symbol, an assertion is being made about an object’s existence which fulfills a criteria, which is true in this case since both $x = +\sqrt{2}$ and $x = -\sqrt{2}$ satisfy this condition.

Sometimes for readability, some authors will use the abbreviation for **such that** (*s.t.*) :

$$\exists x \in \mathbb{R} \text{ s.t. } x^2 = 2$$

For more information on the real set and the belongs to notation see the Collections notebook

```
[14]: # there exists x in R such that x^2 = 2
x = math.sqrt(2)

print(f'x = {x}')
print(f'x^2 = {x ** 2}')
print(f'x^2 == 2: {math.isclose(x ** 2, 2)}')
```

```
x = 1.4142135623730951
x^2 = 2.0000000000000004
x^2 == 2: True
```

There exists uniquely

When the existential quantifier symbol is followed by an exclamation point, it means there exists a **unique** object that fulfills a given criteria: $\exists!$

For example: there exists a unique real number x whose square equals 0

$$\exists! x \in \mathbb{R}, x^2 = 0$$

which is true in this case since only $x = 0$ satisfies this condition.

For more information on the real set and the belongs to notation see the Collections notebook

```
[15]: # there exists exactly one x in R such that x^2 = 0
X = [-2, -1, -0.5, 0, 0.5, 1, 2]
```

```
for x in X:
    print(f'x = {x:5}, x^2 = 0: {x ** 2 == 0}')

solutions = [x for x in X if x ** 2 == 0]
print(f'\nsolutions: {solutions} (exactly one)')
```

```
x =    -2, x^2 = 0: False
x =    -1, x^2 = 0: False
x =   -0.5, x^2 = 0: False
x =     0, x^2 = 0: True
x =    0.5, x^2 = 0: False
x =     1, x^2 = 0: False
x =     2, x^2 = 0: False
```

```
solutions: [0] (exactly one)
```

Combining quantifiers

The for all $\forall$ and exists $\exists$ notations can be combined to denote complex statements.

For example: For all x in the real number set, there exists at least one real number y such that $x + y = 0$

$$\forall x \in \mathbb{R}, \exists y \in \mathbb{R}, x + y = 0$$

This statement means that if we were to pick **any** real number x , we can find at least one number y so that we get $x + y = 0$. We know this to be a True statement since we can always find a number $y = -x$

For more information on the real set and the belongs to notation see the [Collections notebook](#)

```
[16]: # for all x in R, there exists y in R such that x + y = 0
X = [random.random() for _ in range(5)]
```

```
for x in X:
    y = -x
    print(f'x = {x:.4f}, y = {y:.4f}, x + y = {x + y}')
```

```
x = 0.6097, y = -0.6097, x + y = 0.0
x = 0.2906, y = -0.2906, x + y = 0.0
x = 0.0614, y = -0.0614, x + y = 0.0
x = 0.8364, y = -0.8364, x + y = 0.0
x = 0.4357, y = -0.4357, x + y = 0.0
```

Note: The statement order is very important since it evolves logically and combines to form a

logical assertion. The above example was well ordered but consider the following example:

$$\exists y \in \mathbb{R}, \forall x \in \mathbb{R}, x + y = 0$$

In this case we make an assertion that there exists a real number y which will have the property $x + y = 0$ for any real number x . Such a real number y does not exist and so this assertion is **False**.

```
[17]: # there exists y in R such that for all x in R, x + y = 0
      # this is False: no single y works for every x
      X = [1.0, 2.0, 3.0]

      for y in X:
          works_for_all = all(x + y == 0 for x in X)
          print(f'y = {y}: x + y = 0 for all x? {works_for_all}')

      print(f'\nfound a universal y? {any(all(x + y == 0 for x in X) for y in X)}')

y = 1.0: x + y = 0 for all x? False
y = 2.0: x + y = 0 for all x? False
y = 3.0: x + y = 0 for all x? False

found a universal y? False
```